Harpocrates++: Automated Functional Program Generation Against CPU Faults and Silent Data Corruptions

Nikos Karystinos, George-Marios Fragkoulis, Odysseas Chatzopoulos, and Dimitris Gizopoulos, *University of Athens, 157 84, Athens, Greece*

Sudhanva Gurumurthi, *Advanced Micro Devices Inc., Austin, TX, 78741, USA*

Several hyperscalers have recently disclosed the occurrence of silent data corruptions in their system fleets, sparking concerns about the severity of known and the existence of unidentified root causes of faults in CPUs. These incidents reveal that CPUs may generate incorrect results due to latent manufacturing defects, variability, marginalities, bugs, and aging. To tackle this, we present Harpocrates, an automated methodology for the generation of short, constrained-random functional test programs that maximize fault detection in target CPU structures and can be employed at different stages of the system lifecycle. Harpocrates adopts a hardware-model-in-the-loop approach, iteratively refining the generated test programs via a detailed simulation-based microarchitecture engine that models and grades for multiple fault types. Harpocrates adapts to various program generators, instruction set architectures, microarchitectures, and fault types. Our results on seven important CPU structures show that Harpocrates outperforms open source test suites in fault detection capability while attaining much shorter generation times.

As modern CPUs grow increasingly complex and powerful, their ability to execute demanding workloads and process large data volumes becomes more critical. This complexity raises the likelihood of unexpected faults during execution, increasing the risk of data corruptions that may cause incorrect results, information loss, or system crashes.[1,2,3] Data corruptions may stem from hardware defects and marginalities (e.g., temperature), particle strikes, design bugs, or malicious attacks.[4] The risk has intensified as CPU designs scale in complexity.[3] Traditional verification and testing approaches often fail to capture all potential root causes of silent data corruptions (SDCs). Shrinking technology nodes, growing chip counts, and emerging failure mechanisms impose new reliability challenges. Marginal or degrading faults[5] often escape conventional manufacturing tests.

Hyperscaler reports now reveal alarmingly high *defective parts per million* (*DPPM*) rates for CPUs, approaching 1000 DPPM.[1,2,3] Acceptable DPPM levels vary: under 10 DPPM for safety-critical systems, and a few hundred DPPM for cloud or high-performance computing. The high DPPM reports urge the community to verify these rates and investigate their root causes. Lowering the DPPM of CPUs remains vital as deployments scale.

These defectivity disclosures and the emerging fault modes extend the traditional scope of SDC investigations beyond memory and input/output to the core of computation itself. While CPUs have historically been protected against soft errors, using techniques, such as architectural vulnerability factor modeling,[6] new defect mechanisms[5] challenge the assumption

Digital Object Identifier 10.1109/MM.2025.3640385
Date of publication 8 December 2025; date of current version 25 February 2026.

that transient faults alone dominate vulnerability. Consequently, methodologies that efficiently quantify and address latent defects, such as Deutsch et al.,[7] who provide a methodology for estimating vulnerability to timing faults, as well as optimization flows for effective tests[8] or test parameters, as shown in Shamsa et al.,[9] are increasingly important to effectively mitigate data corruption.

CPU fault detection through the execution of programs composed of normal instructions has been studied extensively. This method is known variously as *functional testing* or *native mode testing*. Unlike structural testing methods employed at manufacturing time, placing the CPU in *scan* mode, functional testing detects hardware faults that affect the CPU during normal operation.

ANY PRACTICAL FUNCTIONAL-TEST FRAMEWORK MUST THEREFORE BE HARDWARE-AWARE, CONSIDER THE FULL SYSTEM STACK, AND DELIVER CONCISE PROGRAMS WITH HIGH FAULT-DETECTION CAPABILITY.

Environmental conditions, such as temperature and humidity, affect electronic devices, with core temperature identified as a trigger for SDCs. Previous research has explored the generation of stress tests via genetic algorithms in different contexts and objectives: most often maximization of power consumption and dI/dt voltage noise, or stress viruses for soft errors. Most of these approaches aim for *worst-case instruction sequences* only, which naturally results in a few distinct instructions executed in a tight loop.

Within the broader context of CPU fault screening, these methods serve primarily as a prologue, establishing extreme stress conditions to trigger intermittent faults arising from marginal defects.[4] Subsequently, targeted functional tests optimized specifically for each hardware structure can efficiently detect and expose these faults. Our methodology precisely addresses this second phase.

We present *Harpocrates*,[a,b] a novel methodology and unique toolchain for the automatic generation and grading of effective functional test programs. Harpocrates is aligned with related industry-led efforts, which present functional test program generation frameworks for CPUs. The goal is to maximize the *coverage* of the CPU hardware (go through as much of the hardware as possible) by generating effective but short test programs. By monitoring the CPU's behavior for irregularities (erroneous outputs or crashes) during the execution of these programs, hardware faults can be identified and faulty CPUs isolated. Therefore, an efficient method for generating functional test programs should ideally be *automated* and *hardware-aware*.

For Harpocrates (or any other approach) to be practically plausible, some important challenges should be addressed:

1. *Dealing with modern CPU design and deployment:* Modern CPUs are complex, with multiple interacting layers of hardware and software, and therefore all layers should be considered.
2. *Targeting specific hardware structures:* Most types of hardware faults tend to be localized in specific structures. Aiming to be generically applicable, existing frameworks are hardware-agnostic and do not grade tests on specific CPU microarchitectures. Localizing the fault is important for root cause analysis of defective CPUs and provides valuable field data for driving design improvements in the next generations.
3. *Generating efficient functional test programs:* Both for root causing (CPU designer) and quick detection (CPU user), the programs should have relatively short execution times. Datacenter providers aim for near zero fleet downtime. Test programs (on demand or periodically executed) should minimally affect downtime.

Any practical functional-test framework must therefore be hardware-aware, consider the full system stack, and deliver concise programs with high fault-detection capability. Our framework, *Harpocrates*, meets these requirements and addresses the aforementioned challenges by:

- Employing a novel constrained-random program generator and mutator called *MuSeqGen* (*Mutator and Sequence Generator*). *MuSeqGen's* code generator is built on MicroProbe.[10] *MuSeqGen* extends the instruction set architectures (ISA) support to x86-64 and offers versatile utilities for configurable random generation. *MuSeqGen* integrates a mutation engine that alters or combines generated sequences to generate refined variants of programs (challenges #2 and #3).

[a]This article is an extension of the International Symposium on Computer Architecture 2024 article.[8]
[b]https://en.wikipedia.org/wiki/Harpocrates

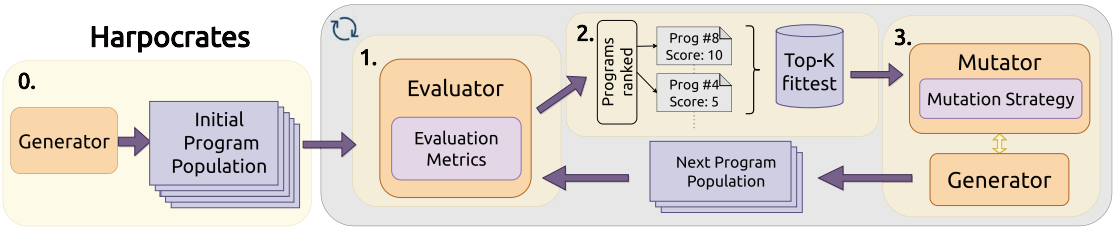

FIGURE 1. The Harpocrates architecture and complete flow of operation during program refinement.

- Increasing the generated programs' detection capability of hardware faults by employing a *hardware-in-the-loop* approach: a detailed CPU model using the gem5 simulator.[c] gem5, and the tooling built around it, is a key piece of Harpocrates in grading and refining the generated programs (challenges #1, #2, and #3).
- Proposing a feedback-based automated methodology for the generation of functional programs. The methodology comprises three distinct components: 1) the Generator, 2) the Mutator, and 3) the Evaluator, which together construct the full Harpocrates program generation loop (challenges #1 and #2).

HARPOCRATES' SYSTEM ARCHITECTURE

Overview

Harpocrates consists of the main components shown as orange boxes in Figure 1, which also depicts their interactions. We will elaborate on the specific tools we have implemented and used in the following section. Different tools that provide a similar functionality can be substituted in each spot.

The three main components are:

- *Generator:* This component generates valid functional test programs. It also bootstraps the iterative refinement process by producing an initial test program population.
- *Mutator:* Operating alongside the Generator, the Mutator alters the current set of programs or combines them to craft new variants.
- *Evaluator:* Acting as the driving force of the system, the Evaluator assesses all generated programs against a predefined metric (an objective function that we are optimizing for). Programs that perform best under this metric (fittest) are retained for subsequent mutation iterations, thereby directing the evolution of test programs toward those that optimize the objective function.[d]

Harpocrates' flow closely resembles that of a genetic algorithm. We manage a collection of programs (i.e., the *population*). These programs undergo combination (akin to *genetic crossover*) and modification (*mutation*). The subsequent generation's population is then determined by an evaluation metric that serves as a *fitness function*. Existing systems like *Silifuzz* are built around similar principles. Moreover, recent work by Shamsa et al.[9] shows that reinforcement learning can be applied to optimize the parameters of existing tests with respect to their fault-detection capability. Harpocrates' approach is more general and perhaps orthogonal to the aforementioned study, as it can generate any set of instructions—there is no predetermined test or "algorithm" whose inputs we optimize—and can adapt to any fault model and evaluation function. Perhaps reinforcement learning is worthwhile to explore as an add-on to our methodology for further optimizing the operand allocation for Harpocrates' optimal instruction sequences; this is currently done via a static policy.

Flexibility and Uses of Harpocrates

The ISA, microarchitecture, optimization, and fault model parameters in Harpocrates can be flexibly adapted as follows:

- any ISA supported by the Generator, Mutator, and Evaluator components can be used
- any microarchitectural structure of a complex out-of-order CPU can be analyzed
- any "quality" metric can be used to guide the iterative refinement of the functional test program (hardware-aware metric tailored to a particular CPU structure)
- any fault model can be used for the evaluation of the final fault detection capability of programs.

[c]https://www.gem5.org/

[d]Different metrics can be used depending on the target structure, the fault model of interest, or other required parameters of the generated programs.

Any combination of the above can be chosen to match the requirements of a particular use case of Harpocrates. A short list of potential use cases is as follows:

- Fast periodic scan of a CPU fleet (the Ripple, in production testing mode, discussed in Dixit et al.[11]): In this case, *Harpocrates* can be constrained to generate programs of certain short duration (cycles or instructions), maximizing their fault detection capability under this constraint.
- Comprehensive scan of CPUs fleet (the Fleetscanner, out of production testing mode discussed in Dixit et al.[11]): In this case, Harpocrates is set to reach a certain (very high) fault detection capability without an execution time constraint.
- Electrical and environment conditions screening before silicon is shipped: Silicon designers look for marginal defects[4,5] that produce faults under certain conditions; when conditions require focus on a particular fault type or structure, Harpocrates can be configured for the purpose.

INSTANTIATING HARPOCRATES: TOOLS, METRICS, AND FAULT MODELS

Tools

Stepping through the components in the order they were presented in the overview of the previous section, the Generator and Mutator are mapped to our code generator and mutator *MuSeqGen*. For the purposes of this study, we have configured *MuSeqGen* for x86-64 assembly program generation and mutation. Mutation was experimentally found to be optimal with respect to convergence when performed via random instruction replacement. More specifically, all instances of instruction *A* are replaced by a randomly sampled instruction *B*. At a high level, the generator constructs programs by emitting sequences of instructions that satisfy basic correctness constraints (valid opcodes, valid register, immediate and memory operands). The operands are selected based on a static policy. For example, immediate operands are always sampled randomly from their whole range (e.g., any 16-bit number for a 16-bit immediate). The mutation operator then explores the neighborhood of a candidate program by replacing all instances of one instruction kind with another randomly sampled kind, a coarse-grained mutation that empirically balances exploration with maintaining program validity. We refer the interested reader to the original Harpocrates article[8] for an in-depth description of the architecture and inner workings of *MuSeqGen*.

The final and arguably most critical component in the pipeline is the Evaluator. In our concrete pipeline, our custom version of gem5 assumes the role of the Evaluator. We have enriched the base microarchitectural modeling engine of gem5 with various analyses and metrics, allowing us to construct hardware-aware fitness functions that are highly targeted to specific hardware structures. Optimizing for suitable fitness functions allows Harpocrates to converge to programs with high detection capability in the target component.

Coverage, Detection, and Fault Models

For the purposes of this study, we shall refer to the hardware-aware fitness functions we construct as *hardware coverage* or simply *coverage*. We define this as follows: Hardware coverage is any objective (fitness) function tied to a specific CPU hardware structure that is expected to correlate well with the fault detection capability of functional programs targeting the structure. Coverage is essentially a proxy of the ability of a program to both activate (*sensitize*) faults and make them observable (*propagate*).

gem5's purpose in this study is twofold: First, our enriched gem5 is used to measure the hardware coverage (fitness function) we have tailored to the target component and fault type. Second, we utilize our fault injection infrastructure built around gem5 to accurately measure the *detection capability* of programs for a specific fault type and hardware structure. Our infrastructure can inject *transient* (single bit-flips, overwritable; usually particle strike induced), *permanent* (stuck-at faults, non-overwritable), or *intermittent* (exist for a specific number of cycles and can behave as permanent or transient) faults in more than 50 microarchitectural targets. Further, we can inject gate-level stuck-at faults in the functional units of the CPU. The *fault detection capability* of programs is assessed via *statistical fault injection* (SFI). By conducting thousands of injection experiments in the selected hardware structure, we obtain a statistically significant, reliable, "golden" measure of a program's ability to detect faults.

An important observation here is to highlight the *impracticality* of a flow that would refine programs by directly measuring detection capability through SFI. Such a flow would require several hours for just a single iteration, in contrast to the fast iteration speed (often in the range of seconds or even subseconds) offered by Harpocrates. This is precisely the reason we have constructed proxy metrics—which are measurable in a single program execution—to optimize instead.

To evaluate Harpocrates we have chosen *seven* key hardware structures comprising both bit-array components (memories, queues) and functional unit

components (adders, multipliers). The former are studied for transient faults (single-bit flips applied uniformly across the whole program execution), while the latter are modeled at the gate-level and injected with permanent stuck-at faults at randomly chosen gates.

To guide the Harpocrates methodology for array-based hardware components (caches, regfiles, queues), we focus on a hardware coverage metric related to transient faults. *Architecturally correct execution* (*ACE*) analysis[6] identifies the fraction of bits (over time) crucial for correct execution thus estimating the program's vulnerability to transient faults. The ACE lifetime analysis computes, in each cycle (incorporating the notion of time which is needed for transient faults), the bits that are necessary for ACE and labels them as ACE. We use the ACE lifetime analysis result (the "vulnerability"; ranging from 0 to 1 or 0% to 100%) as our *hardware coverage* measurement for the bits of a CPU array structure when a program runs.

The ACE lifetime analysis metric is not suitable for other fault types, such as permanent faults in arithmetic units. For those cases, we define the *input bit ratio* (*IBR*) as a more fitting coverage metric. IBR measures the total input bits to a unit across program execution divided by the theoretical maximum count of input bits, assuming the unit's inputs are saturated at every cycle of program execution.[e]

EXPERIMENTAL SETUP

Open Source CPU Testing Frameworks

Aiming to drive community-wide efforts in improving defects detection in CPUs, some open source frameworks, contributed by the industry, have been publicly released. Many of these test frameworks have been released to the community through the broader industry-wide effort in the context of the Open Compute Project (OCP) and are summarized in "OCP, Hardware Management Talks."[12] Our aspiration for Harpocrates is to be a major contributor to these efforts. In the "Experimental Evaluation" section, we present results from the open source tests these suites publicly offer and compare them with Harpocrates.

SiliFuzz

SiliFuzz[f] employs fuzzing on (hardware-like) software proxies like CPU simulators and disassemblers. It uses software-coverage metrics to construct a diverse input set, known as *the corpus*. High code coverage in proxies is conjectured to uncover interesting CPU behavior and detect defects. To achieve statistically significant results in our fault injection-based evaluation, instructions from multiple snapshots (maximum of 100 bytes of binary code each) are aggregated into a single 10 K instructions test (the typical size of our methodology's test programs).

OpenDCDiag

OpenDCDiag[g] is an open-source project designed to identify defects and bugs in CPUs. The open-source tests provided include compression, cryptographic operations, matrix multiplication, and singular value decomposition. These algorithms are sensitive to data corruption; corruption in their inputs or intermediate results is highly likely to result in corruption in the output data. In our evaluation of OpenDCDiag, we consider each test separately, and evaluate a single execution of the full test. Input sizes are configured so that no test exceeds 100 million cycles of execution.

Hardware Components and Fault Types

To demonstrate the effectiveness of Harpocrates, we present results for seven key hardware structures: 1) physical (integer) register file (IRF), 2) L1 data cache (L1D), 3) load-store queue (LSQ), 4) integer adder, 5) integer multiplier, 6) SSE floating-point (FP) adder, and 7) SSE FP multiplier. We intentionally selected the first five hardware structures for a fair comparison of Harpocrates with SiliFuzz and OpenDCDiag (which are hardware-agnostic) and to a general-purpose benchmarks suite (MiBench[h]) since all of them are expected to offer reasonably high utilization of these critical components. The two SSE FP arithmetic units were chosen to complete the picture; recent reports from hyperscalers clearly identify FP units along with their integer counterparts (in scalar and vector hardware) as very likely sources of SDCs in large fleets.[3] Harpocrates can produce effective programs for any other hardware structure.

Setting the Baseline

To establish a fair baseline, we evaluated the effectiveness of the open source frameworks (SiliFuzz and OpenDCDiag) using the same key metrics that drive Harpocrates: *hardware coverage* (measured via ACE and IBR) and, more importantly, *fault detection*

[e]IBR can be <1 due to several factors. For example, a 64-bit arithmetic unit may not be utilized in every cycle. Moreover, its inputs may not always be 64-bits wide.

[f]https://github.com/google/silifuzz

[g]https://github.com/opendcdiag/opendcdiag

[h]https://github.com/embecosm/mibench

capability assessed through SFI. We also report these metrics for 12 general-purpose benchmarks from the MiBench suite.

Figure 2 gathers results for all bit-array components (IRF, L1D, LSQ) injected with transient faults. The IRF detection capability is very low (less than 5%) across most of the programs considered, with few outliers in MiBench and SiliFuzz that marginally pass the 5% mark. Regarding the L1D cache, the detection capability across all three frameworks is significantly higher than the IRF, reaching more than 80% for a single OpenDCDiag program. Finally, regarding the LSQ—more specifically the *SQ data* field, which is injected with faults—although ACE measures high coverage for some benchmarks, detection capability is low, with few OpenDCDiag programs achieving 20% at best, while the rest hover around 0–1%.

Figure 2 clearly shows that there is significant room for improvement and underlines the objective of Harpocrates. The respective results for functional units are omitted from this text due to space requirements. You can examine them in detail in the original article.[8]

EXPERIMENTAL EVALUATION

Hardware used was AMD EPYC™ 7402 24-Core CPU with 128GB of DDR4 RAM.

Performance Evaluation

Harpocrates' generation and evaluation speed can be compared to SiliFuzz, which is the only alternative automated methodology available for this purpose. We calculated the effective instruction generation rate for both frameworks and found that Harpocrates' (runnable and evaluated) instruction generation rate is 30 times that of Silifuzz. For details on the calculation, see the original article.[8]

Harpocrates Parameters and Convergence

The graphs of Figure 3 show the coverage (light pink dots) and detection (red crosses) as the optimization loop progresses. For details on the parameterization of each optimization loop, we refer the interested reader to the original article.[8]

A key observation from Figure 3 confirms the central assumption of our methodology: Increasing test coverage via Harpocrates directly improves detection capability. Our hardware-aware coverage metrics, tailored to fault types and structures, show strong correlation with detection, validating Harpocrates' core principle of using *accurate*, *quantitative feedback* to evolve highly specialized, microarchitecture-targeted programs. This automated, hardware-aware evaluation distinguishes Harpocrates from methods that rely on expert intuition or hardware-agnostic metrics, like SiliFuzz.

Another important finding is the disparity in detection difficulty. Transient faults in bit-array components are significantly harder to detect than permanent faults in functional units. Transients are quickly overwritten, and components like the IRF, LSQ, and L1 cache have complex dynamic behaviors, making them harder to cover. Functional units, by contrast, respond predictably to specific instructions. This is reflected in Harpocrates' convergence: ~1000 refinement loops

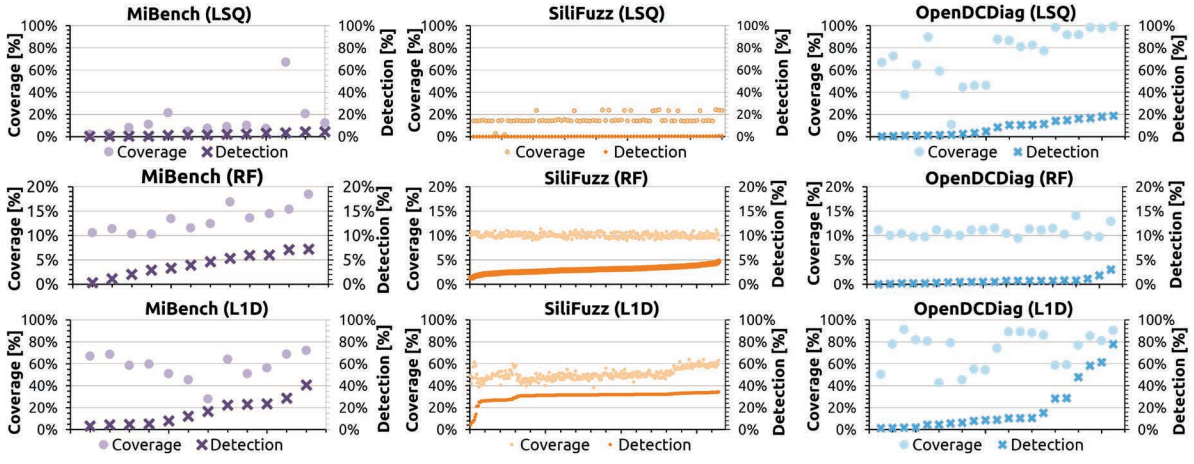

FIGURE 2. Coverage (ACE) (light dots, left *y*-axis) and detection (transients) (dark crosses, right *y*-axis) for bit-array components.

suffice for functional units, while the cache requires 2000+, and IRF/LSQ peak near 5000 iterations. Detecting transient faults in bit arrays is thus inherently more challenging than addressing permanent faults in functional units.

Fault Detection Full Comparisons

This section aggregates and compares the fault detection capability for all frameworks for the seven hardware components of this study (Figure 4). The results from the open source tests (Silifuzz and OpenDCDiag) and the MiBench workloads are shown to put things in perspective and showcase how Harpocrates can generate tests that adapt to different components and fault modes, providing excellent detection capability. For further commentary on these results, please refer to the original article.[8]

The detection capability of Harpocrates' tests is well supported by our coverage metrics. For bit array components, ACE defines an upper bound for detection, and as shown in Figure 3, our generated programs closely approach this bound, indicating minimal software masking due to careful generator parameterization.

A similar trend is seen for functional units: While IBR measures unit utilization rather than detection, the strong correlation between the two suggests our programs effectively propagate unit outputs. In contrast, workloads from MiBench, SiliFuzz, and OpenDCDiag exhibit weaker correlations, reflecting substantial software masking.

Our approach also excels in detection speed. Although a MiBench program eventually matches our 99% detection for the integer adder, it requires more than 11 million cycles. Harpocrates' programs reach the same level in only 50,000 cycles—about 220 times faster—and achieve similar gains for the multiplier. Compared to SiliFuzz's best test, which detects just

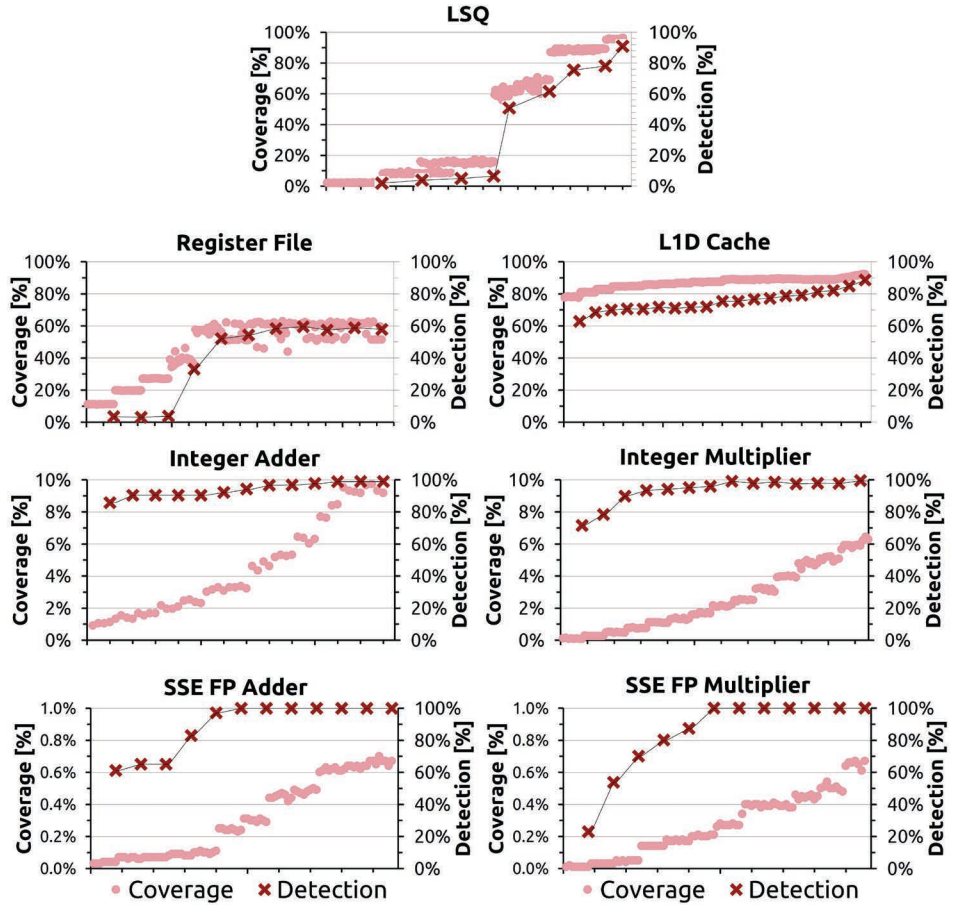

FIGURE 3. Coverage (light dots, left *y*-axis) and detection (dark crosses, right *y*-axis) for all components, measured across Harpocrates optimization.

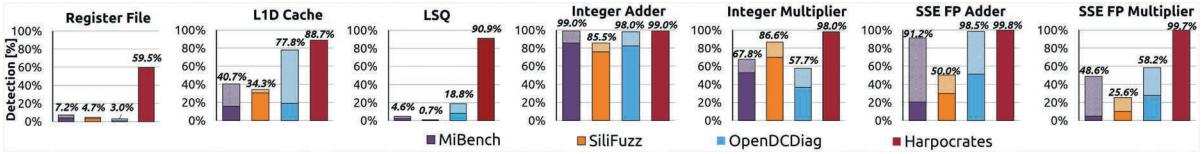

FIGURE 4. Maximum (top of bars) and average (middle of bars when shown) detection values for all hardware structures.

86.6% of faults in comparable time, our approach attains 99.5%.

The Impact of Randomness

The Mutator (*MuSeqGen*) in the Harpocrates instantiation we have presented mutates at the level of the instruction sequence. The operands of the selected instructions (immediates, memory references, registers) are resolved through a carefully designed static policy (more details in the original article[8]). Of these operands, the immediates, as well as the initial state, of all registers are resolved randomly.

Naturally, the question arises: *Given the same instruction sequence, what is the impact of different random seeds on its detection capability, i.e., different immediate operands as well as initial state?*

To answer this question, we gathered the best programs for each component produced by Harpocrates and generated 50 variant programs of their instruction sequences, each with a different random seed, which we evaluate in what follows.

Figure 5 shows the resulting distributions for the 50 variants as swarm plots. Most of the examined components display very low variance in detection capability. Specifically, in Figure 5 the variants of the best instruction sequence produced by Harpocrates for the *integer adder* (blue), *SSE FP adder* (green), *SSE FP multiplier* (red), and the *load-store queue* (pink)

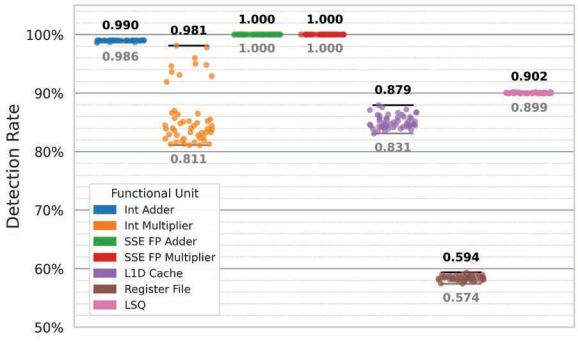

FIGURE 5. Detection rate of 50 immediate-data variants of the best instruction sequences for each component. Maximum (top line, black) to minimum (bottom line, gray).

random seed used to resolve immediates, and the initial state, seem to have minimal impact on the final detection capability of the variant programs (<1%). Looking at the *integer register file* (brown) and the *L1 data cache* in Figure 5, there is slightly more variance; ≈2% and ≈5%, respectively. The component exhibiting the highest detection variance among the programs is the *integer multiplier* (≈17%).

In summary, the results show low or very-low variance in detection capability for almost every component. From an evolutionary perspective, it makes sense for the program population to evolve in a way that minimizes fitness function losses due to the randomness present during program generation. The component that exhibits higher variance is the *integer multiplier*. There, initial state and the resolution of immediate values contained in the program sequence seem to impact the final detection capability measured for the program variant more significantly. This effect is partly explained by a quirk of x86 integer multiplication: Certain instruction variants write results to predefined registers, overwriting the register state on each execution and potentially masking erroneous results.

Evaluating Smaller Subsequences

So far, Harpocrates has generated programs with a fixed instruction budget, which varied for each component. Bit-array components can be large in size (e.g., the L1D cache) or exhibit highly dynamic behavior depending on the program (e.g., the LSQ or IRF). Consequently, adequately exercising these components calls for program sequences of sufficient length. The instruction budgets assigned to Harpocrates for these components reflect this fact, which we have also confirmed experimentally. To give a simple example, one cannot expect to fully "cover" a 32-KB cache with 100 instructions.

For functional units, however—where the faults injected are permanent and occur at the gate level—we may pose the following question: *Can such faults be detected effectively with shorter sequences?* This idea relates to Ripple mode testing,[11] in which in-production systems are screened. In this mode, the objective is to

minimize the impact on machine availability; hence, short and effective sequences are preferred.

To explore this, we designed the following experiment: For each functional unit, select one instruction sequence from the best generation of programs produced by the Harpocrates optimization loop. Then, take variously sized prefixes ("slices") of these sequences, regenerate full programs, and evaluate their detection capability. Harpocrates takes care to finalize sequences of any size with a short epilogue, which pushes program state to observable output; for more details please refer to the original article.[8]

We configured the experiment with six "slice" or prefix ratios: 0.01×, 0.1×, 0.25×, 0.5×, 1.0× (the original sequence/program), and 2.0× (a duplication of the original program). The results are presented in Figure 6. The *y*-axis corresponds to the different functional units, while the *x*-axis represents the sequence prefix sizes, ranging from smaller prefixes on the left to larger prefixes on the right. Detection rates are shown in the colored boxes as percentages in Figure 6, with dark green shades indicating higher detection. Detection performance remained virtually unchanged up to a slice ratio of 0.1×, meaning that permanent gate-level faults could be adequately detected using only the first 10% of program instructions (hundreds of instructions instead of thousands). At a 0.01× ratio (tens of instructions) the results varied: *integer adder* and *multiplier* showed a slight decrease in detection compared to 0.1×, the *SSE FP adder* showed no drop, but the *SSE FP multiplier*'s detection rate dropped to 0%.

In summary, in our setup, permanent gate-level faults appear to be detectable with only a few hundred instructions. This may seem surprising, but it follows from the permanence of such faults: They are observable at all times and under all conditions. In practice, however, fault models for functional units may differ significantly; a more realistic approximation might involve intermittent or marginal events related to path delays (e.g., triggered by temperature or voltage fluctuations), in which case longer sequences would likely be more effective.

CONCLUSION

Harpocrates introduces a hardware-aware, feedback-driven methodology for automatically generating test programs to detect CPU defects. By integrating coverage metrics closely aligned with specific fault models and microarchitectural structures, it consistently produces high-quality tests that outperform existing frameworks in both detection capability and efficiency. Our results confirm that accurate, iterative coverage feedback enables rapid convergence on highly effective instruction sequences, making Harpocrates a practical and adaptable solution for defect detection throughout the lifetime of a system.

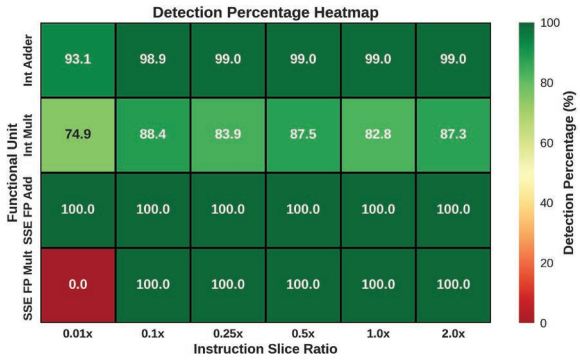

FIGURE 6. Detection rate (permanent gate-level faults) for prefixes (slices) of the best instruction sequences in FUs.

ACKNOWLEDGMENTS

The University of Athens team authors have been supported by the European Union through the EuroHPC JU Grant 101202459 (DARE-SGA1), the Horizon Europe programme Grants 101093062 (Vitamin-V) and 101070238 (Neuropuls), and the Chips-JU grant 101097224 (REBECCA). Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them. Research is also supported by the Open Compute Project foundation, Advanced Micro Devices (AMD), and Meta through research gifts on silent data corruptions. AMD, the AMD Arrow logo, and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

REFERENCES

1. H. D. Dixit et al., "Silent data corruptions at scale," 2021, *arXiv:2102.11245*.
2. P. H. Hochschild et al., "Cores that don't count," in *Proc. Workshop Hot Topics Operating Syst. (HotOS)*, New York, NY, USA: ACM, 2021, pp. 9–16, doi: 10.1145/3458336.3465297.
3. S. Wang, G. Zhang, J. Wei, Y. Wang, J. Wu, and Q. Luo, "Understanding silent data corruptions in a large production CPU population," in *Proc. 29th Symp. Operating Syst. Princ. (SOSP)*, New York, NY, USA: ACM, 2023, pp. 216–230, doi: 10.1145/3600006.3613149.

4. P. G. Ryan, I. Aziz, W. B. Howell, T. K. Janczak, and D. J. Lu, "Process defect trends and strategic test gaps," in *Proc. Int. Test Conf.*, 2014, pp. 1–8, doi: 10.1109/TEST.2014.7035276.
5. S. Gurumurthi, V. Sridharan, and S. Gurumurthy, "Emerging fault modes: Challenges and research opportunities," *SIGARCH*, Jul. 17, 2023. [Online]. Available: https://www.sigarch.org/emerging-fault-modes-challenges-and-research-opportunities/#
6. S. S. Mukherjee, C. Weaver, J. Emer, S. K. Reinhardt, and T. Austin, "A systematic methodology to compute the architectural vulnerability factors for a high-performance microprocessor," in *Proc. 36th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Piscataway, NJ, USA: IEEE Press, 2003, pp. 29–40, doi: 10.1109/MICRO.2003.1253181.
7. P. W. Deutsch, V. Q. Ulitzsch, S. Gurumurthi, V. Sridharan, J. S. Emer, and M. Yan, "DelayAVF: Calculating architectural vulnerability factors for delay faults," in *Proc. 57th IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 231–245, doi: 10.1109/MICRO61859.2024.00026.
8. N. Karystinos, O. Chatzopoulos, G.-M. Fragkoulis, G. Papadimitriou, D. Gizopoulos, and S. Gurumurthi, "Harpocrates: Breaking the silence of CPU faults through hardware-in-the-loop program generation," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, 2024, pp. 516–531, doi: 10.1109/ISCA59077.2024.00045.
9. M. Shamsa et al., "Improved silent data error detection through test optimization using reinforcement learning," in *Proc. IEEE Int. Rel. Phys. Symp. (IRPS)*, 2025, pp. 8C.1-1–8C.1-5, doi: 10.1109/IRPS48204.2025.10983294.
10. R. Bertran, A. Buyuktosunoglu, M. S. Gupta, M. Gonzalez, and P. Bose, "Systematic energy characterization of CMP/SMT processor systems via automated micro-benchmarks," in *Proc. 45th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2012, pp. 199–211, doi: 10.1109/MICRO.2012.27.
11. H. D. Dixit, L. Boyle, G. Vunnam, S. Pendharkar, M. Beadon, and S. Sankar, "Detecting silent data corruptions in the wild," 2022, *arXiv:2203.08989*.
12. "2023 OCP hardware management tech talks," *Open Compute Project*, May 17, 2023. [Online]. Available: https://www.opencompute.org/events/past-events/2023-ocp-hardware-management-tech-talks

NIKOS KARYSTINOS is a Ph.D. student and a member of the Computer Architecture Lab at the Department of Informatics and Telecommunications of the University of Athens, 157 84, Athens, Greece. His research interests include microarchitectural simulation, hardware reliability, and program generation. Karystinos received his MSc degree in computer science with a specialization in computer systems: software and hardware from the University of Athens. Contact him at n.karystinos@di.uoa.gr.

GEORGE-MARIOS FRAGKOULIS is a Ph.D. student and a member of the Computer Architecture Lab at the Department of Informatics and Telecommunications of the University of Athens, 157 84, Athens, Greece. His research interests include performance, reliability, and virtualization. Fragkoulis received his BSc degree in computer science from the University of Athens. Contact him at gm.fragkoulis@di.uoa.gr.

ODYSSEAS CHATZOPOULOS is a Ph.D. student and a member of the Computer Architecture Lab at the Department of Informatics and Telecommunications of the University of Athens, 157 84, Athens, Greece. His research interests include reliability analysis of heterogeneous system architectures from edge devices to hyperscale systems. Chatzopoulos received his BSc degree in computer science from the University of Athens. Contact him at od.chatzopoulos@di.uoa.gr.

DIMITRIS GIZOPOULOS is professor of computer architecture at the Department of Informatics and Telecommunications of the University of Athens, 157 84, Athens, Greece. His research interests include the complex interactions among performance, power, and reliability of computing systems built on CPUs, GPUs, and domain-specific accelerators. He serves as associate editor and guest editor for several IEEE and Association for Computing Machinery (ACM) publications and is a member of the steering, organizing, and program committees of international computer architecture, hardware, and systems conferences. Gizopoulos received his Ph.D. degree in computer science and engineering from the University of Athens. He is a Fellow of IEEE, an ACM distinguished member, and a golden core member of the IEEE Computer Society. Contact him at dgizop@di.uoa.gr.

SUDHANVA GURUMURTHI is a Fellow at Advanced Micro Devices, Austin, TX, 78741, USA, where he is currently responsible for research and advanced development in reliability, availability, and serviceability. His research interests include computer architecture, reliability, and energy-efficiency. Sudhanva received his Ph.D. degree in computer science and engineering from Pennsylvania State University in 2005. Contact him at sudhanva.gurumurthi@amd.com.